

Ask-Your-Data: Analytics & Review Intelligence

Capstone Evaluation Report — Team 2

SQL + AI/LLM Analytics Assistant over the Olist Brazilian E-Commerce Dataset

1. Project Summary

This project built a conversational analytics assistant that lets a non-technical user ask a plain-English question about the business and receive an answer grounded in both structured order data and unstructured customer review text. The system consists of three layers: a SQL layer over the Olist e-commerce dataset, an NLP layer providing sentiment scoring and semantic review search, and an LLM layer that translates natural-language questions into SQL. All three layers were integrated into a single Streamlit application.

Team contributions

- Kashmala & Khaula: database design, loading, and core SQL queries; found and fixed two data-quality issues (missing category translations, an over-aggressive deduplication step) and one query-level row-duplication bug.
- Laiba: sentiment analysis (nlptown/bert-base-multilingual-uncased-sentiment) and semantic search (paraphrase-multilingual-MiniLM-L12-v2 + ChromaDB), with both components independently evaluated against real star ratings and tested for retrieval relevance.
- Nimra: text-to-SQL pipeline (prompt engineering + Hugging Face LLM), integration of all three layers, and the deployed Streamlit application.

2. Grading Criteria - Summary

Criterion	What "strong" looks like	Our result
SQL depth	Real joins/aggregations across multiple tables	9 tables loaded, schema fully normalized with enforced foreign keys; all 7 core relationships verified with 0 orphaned rows after 2 data-quality fixes; all 15 text-to-SQL test queries use multi-table joins
Sentiment accuracy	Measured against real star ratings, with a reported figure	84.63% overall accuracy on 449 held-out reviews; 95% negative recall; 97% positive precision
Text-to-SQL correctness	Gets most of ~15 test questions right	15 / 15 correct, with 5 documented fixes along the way
Semantic search quality	Returned reviews are genuinely relevant	Verified relevant, cross-lingual retrieval (English queries correctly matched Portuguese reviews); average cosine similarity distance < 0.42 on domain queries

Integration & demo	All layers work together in one usable interface	Deployed Streamlit app combines SQL + semantic search into one ask_question() function; publicly accessible with auditable SQL, filtered review evidence, and safety guardrails against destructive queries
--------------------	--	---

3. SQL Layer (Kashmala & Khaula)

SQLite was chosen over PostgreSQL to avoid server setup overhead. The database was built and verified in a self-contained Jupyter notebook (khaula_sql_layer.ipynb) capable of rebuilding the full database from scratch and re-running every query end to end, alongside a standalone schema file (sql/schema.sql).

Final verified row counts

- customers: 99,441
- geolocation: 1,000,163
- orders: 99,441
- order_items: 112,650
- order_payments: 103,886
- order_reviews: 99,224 (corrected from an initial, erroneous 98,410, see below)
- products: 32,951
- sellers: 3,095
- product_category_name_translation: 73 (71 original + 2 patched, see below)

Data-quality issues found and fixed

Two genuine data-quality issues were found through direct verification against the live database, rather than assumed correct:

- Missing category translations: 13 products referenced two category names (pc_gamer and portateis_cozinha_e_preparadores_de_alimentos) that were entirely absent from the translation table. English translations for both were added directly in the load script so the fix persists across a full rebuild, rather than as a one-off manual patch.
- Dropped genuine reviews: the load script's drop_duplicates() call on review_id was deleting 814 legitimate reviews. Investigation found 789 review_id values appearing more than once in the raw file (1,603 rows total), but in every case the duplicate rows had different order_id values, the same review text had legitimately been left for more than one order, not a true duplicate. The drop_duplicates() call was removed entirely; order_reviews now loads the full, correct 99,224 rows. This is also why review_id was deliberately not made a primary key in the schema.

After both fixes, PRAGMA foreign_key_check and a manual orphan-count check across all seven declared parent-child relationships were run against the final rebuilt database and returned zero violations / zero orphaned rows.

Query-level bug: row duplication from item-level joins

A second review pass found that the query measuring review score against delivery lateness by product category (order_reviews → order_items → products) was inflating its counts: because a single order can contain items from more than one product/category, each review was being duplicated once per item in its order. Verified directly: 95,601 delivered reviews became 109,125 rows after the join, roughly 14% inflation, traced to 9,803 multi-item orders. The fix rebuilt the query to first collapse to a distinct list of order/category pairs via a common table expression, then join reviews to that list, so each review contributes once per distinct category its order touches rather than once per item.

Notably, this is the same category of bug independently discovered during text-to-SQL testing (Section 6): item-level tables joined to order-level data duplicating rows and skewing aggregates. Two independent parts of the project surfaced the same underlying data-modeling risk, which strengthens confidence that it was a real, worth-fixing issue rather than a one-off mistake.

4. Sentiment Analysis (Laiba)

Sentiment was scored using nlptown/bert-base-multilingual-uncased-sentiment, a transformer model fine-tuned on customer reviews across six languages, chosen specifically to handle informal Portuguese review text natively, including negations, idiomatic logistics phrases (e.g. "atraso na entrega", "produto quebrado"), and morphology without lossy machine-translation preprocessing. The model outputs a discrete 1–5 star prediction, directly comparable to Olist's own 5-point rating scale.

Evaluation methodology

Verified customer star ratings served as ground truth: 1–2 stars mapped to Negative, 4–5 stars to Positive. Neutral 3-star reviews were excluded from the accuracy evaluation, since they frequently contain mixed sentiment (e.g. "product arrived intact but took three weeks") and would introduce noise into a binary ground-truth benchmark. The model was validated against a held-out set of 449 verified reviews.

Measured results

- Overall accuracy: 84.63% (449 reviews).
- Negative (1–2 star) class: 0.66 precision, 0.95 recall, 0.78 F1 (support: 129).
- Positive (4–5 star) class: 0.97 precision, 0.81 recall, 0.88 F1 (support: 320).

The high negative recall (95%) is the operationally important number: in e-commerce review monitoring, failing to flag a genuinely dissatisfied customer (a false negative) carries more business risk than over-flagging a satisfied one, so the model was evaluated with that asymmetry in mind rather than optimizing for raw accuracy alone. The high positive precision (97%) means that when the model does call a review positive, it is very rarely wrong, keeping any positive-sentiment metrics built on top of it trustworthy.

5. Semantic Search (Laiba)

Review text was embedded into 384-dimensional dense vectors using sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2, chosen because keyword matching fails when a search query and a review describe the same problem in different words (e.g. searching "broken packaging" should still surface a review saying "caixa amassada e mercadoria danificada"). Vectors were indexed in a persistent ChromaDB store (chroma_db_reviews/), with each entry enriched with order_id, the verified review_score, and the model's predicted_stars, enabling hybrid filtering such as restricting

semantic matches to $\text{review_score} \leq 2$, which the integration layer (Section 7) uses when a business question implies a problem.

Verified relevance

Retrieval tests reported an average cosine similarity distance below 0.42 for domain-specific operational queries, with sub-second nearest-neighbor search. This was independently confirmed during integration testing: an English query ("delayed delivery and late shipping") correctly retrieved genuine Portuguese-language complaint reviews about non-delivery and missed deadlines, confirming the multilingual embedding model bridges the language gap as intended. A more targeted query ("late delivery complaint product never arrived delayed") reliably surfaced low-star, on-topic complaints, while a broader or under-specified query risked pulling in tangentially related but sentiment-mismatched reviews, which is precisely why the integration layer applies the $\text{review_score} \leq 3$ filter rather than relying on retrieval relevance alone.

6. Text-to-SQL Pipeline (Nimra)

The pipeline uses a Hugging Face-hosted LLM (Llama-3.3-70B-Instruct) with a schema-aware system prompt built dynamically from the live database (via SQLite PRAGMA introspection), so the schema description can never go stale relative to the actual database.

Test results - 15/15 correct

A 15-question test set was designed to span easy single-table lookups, medium multi-table joins, and hard business-logic-heavy questions combining delivery, revenue, and review data.

#	Question	Result	Notes
1	How many total orders are there?	Correct	99,441 orders
2	How many unique customers are there?	Correct (after fix)	Used customer_id instead of customer_unique_id on first attempt; corrected via prompt rule
3	Order statuses and counts	Correct	delivered: 96,478 dominant
4	Average delivery time by state	Correct	Correctly excluded NULL delivery dates
5	Total revenue by category	Correct (after fix)	Initially included freight_value in revenue; corrected via prompt rule
6	Most-used payment type	Correct	credit_card, most common
7	Average review score overall	Correct	4.09 / 5
8	Sellers with highest order count	Correct (after fix)	Initially double-counted multi-item orders; fixed with COUNT(DISTINCT)

9	Average freight value by category	Correct	Heavy/bulky categories correctly ranked highest
10	Categories with most late deliveries	Correct	Standout: combined with review evidence
11	Categories with worst review score	Correct (after fix)	Row-duplication bug from order_items join; fixed with worked example in prompt
12	Delivery delay vs. review score	Correct	Standout: 3-way breakdown (On Time / Late / Cancelled) + review evidence
13	State with highest total revenue	Correct	SP (São Paulo)
14	% of orders delivered late	Correct	8.1% of delivered orders
15	Highest-AOV category + its review score	Correct	Combined two prior fixes correctly without re-prompting

Documented prompt-engineering iterations

Five genuine failures were found and fixed during testing, each traced to a specific root cause and corrected with a targeted prompt rule (in two cases, a soft instruction was insufficient and had to be replaced with a worked SQL example, which proved more reliable):

#	Issue found	Root cause	Fix applied
1	Revenue miscalculated	SUM(price + freight_value) instead of price only	Added explicit revenue definition to prompt
2	Seller order count inflated	COUNT(order_id) counted item rows, not distinct orders	Added COUNT(DISTINCT order_id) rule
3	Category review-score ranking skewed	order_items joined to order_reviews duplicated rows per order	Added a worked example (DISTINCT subquery pattern) to the prompt
4	Undelivered orders miscounted as "on time"	NULL delivery dates evaluated as false in CASE logic	Added explicit NULL-handling rule
5	Customer count inflated	COUNT(DISTINCT customer_id) counts orders, not people	Added rule to use customer_unique_id for person-level counts

7. Streamlit Application (Nimra)

All three layers were combined into a single ask_question() function that (1) generates and runs SQL against the live database, and (2) retrieves supporting customer review evidence via semantic search when

the question implies a qualitative dimension (e.g. delays, quality, complaints). This function powers a Streamlit web application built as the project's user-facing interface.

Application features

- A single text-input box where a non-technical user types a plain-English business question.
- The generated SQL query is displayed alongside the result, so the answer is auditable rather than a black box.
- Results render as a data table for easy scanning.
- A toggle lets the user include or exclude supporting customer review evidence; reviews are automatically skipped for pure counting/statistics questions where they wouldn't add value, and automatically filtered to low-star (≤ 3) reviews when the question implies a problem (e.g. contains "late", "worst", "complaint").
- Each review is shown with its real star rating alongside the review text, so a reader can judge the evidence themselves rather than trust an unlabeled snippet.
- An example-questions panel gives a new user a starting point without needing to guess what the system can answer.

Deployment

The app is deployed publicly via Streamlit Community Cloud, connected to a GitHub repository containing the app code, requirements, and Laiba's ChromaDB vector store. Several practical engineering obstacles were encountered and resolved during deployment, documented here for completeness rather than omitted:

- Database size: the initial SQLite database (106MB) exceeded GitHub's upload limits. Diagnosing this by row count identified the geolocation table (1,000,163 rows, unused by any of the 15 test questions) as the dominant contributor; dropping it and running VACUUM reduced the file to 65.2MB.
- Even at 65.2MB, the file still exceeded GitHub's 25MB browser-upload limit. Rather than force a local Git-command-line workflow, the database was instead hosted on Google Drive, and the app was written to auto-download it on first run if not already present locally, keeping the GitHub repository itself lightweight (code and vector store only) and avoiding any Git tooling requirement.
- Hugging Face Spaces was evaluated as an alternative host but ruled out: Streamlit is no longer a free built-in SDK there (deprecated in favor of Docker-based deployment), and the Docker option was showing as a paid tier on the account used, confirmed directly from the Space-creation screen before switching platforms.
- The Chroma vector store's folder structure required care during upload, the extracted directory nested one level deeper than expected, which silently caused the retriever to connect to an empty placeholder collection rather than erroring outright. This was caught by explicitly checking `collection.count()` before assuming the retriever was working, rather than trusting a lack of errors.

The Hugging Face API token is supplied via Streamlit Cloud's encrypted secrets store rather than hardcoded, and the app enforces a query allowlist (blocking DROP / DELETE / UPDATE / INSERT / ALTER) before executing any LLM-generated SQL against the live database, as a safety measure against destructive queries.

8. Standout Examples

Question: "Which product categories have the most late deliveries?"

SQL result: bed_bath_table (811 late deliveries), health_beauty (776), sports_leisure (584), furniture_decor (535), computers_accessories (503).

Supporting review evidence retrieved for the same question included multiple 1–2 star Portuguese-language reviews describing non-arrival and delivery delays (e.g. a customer reporting 30 days without receiving their order), directly corroborating the SQL finding with real customer voice, exactly the kind of blended answer the assignment's standout criteria call for.

A second standout example, "Is there a relationship between delivery delay and review score?" produced a three-way breakdown (On Time: 4.29 avg score, Late: 2.57, Undelivered/Cancelled: 1.76), showing that non-delivery is rated even more harshly than late-but-completed delivery, backed by matching complaint reviews.

9. Limitations & Honest Notes

SQL layer

- review_id is intentionally not a primary key, since the source data does not guarantee it is unique (confirmed by the 789-value duplicate-with-different-order_id finding, Section 3).
- 610 products have a NULL product_category_name, a genuine missing value in the source data, distinct from an orphaned foreign key. These rows are retained and surfaced as "unknown" via COALESCE rather than dropped.
- Two different revenue definitions exist across the team's queries (price only vs. price + freight_value); both are legitimate depending on the question asked (merchandise revenue vs. total sales value), but this has been flagged for a naming cleanup rather than silently reconciled. The text-to-SQL layer (Section 6) standardizes on price-only, matching the "merchandise revenue" definition.
- The payment-method breakdown query is not filtered to delivered orders, so its totals include payments tied to orders that may later have been canceled, a legitimate framing for "all payment activity" but worth stating explicitly if reused elsewhere.

Text-to-SQL layer

- Soft prompt instructions were not 100% reliable on their own; concrete worked examples in the prompt proved more effective than described rules for preventing recurring SQL patterns (e.g. row duplication from item-level joins), mirroring the exact bug Khaula found independently in the SQL layer's own query 5.6 (Section 3).
- The deployed app's database excludes the geolocation table (1,000,163 rows, unused by any test question) to keep the hosted database under size limits; the full 9-table database with geolocation intact, as built and verified by Khaula, is preserved separately as the project's primary database deliverable.

Sentiment layer

- Overall accuracy (84.63%) is pulled down by negative-class precision (0.66); the model over-flags some positive/neutral reviews as negative - while negative recall stays high (0.95), which is the intentional, business-motivated trade-off described in Section 4.
- Neutral (3-star) reviews were excluded from the formal accuracy evaluation; sentiment on neutral reviews specifically has not been separately validated.

10. Conclusion

The assistant meets every criterion in the grading rubric with measured, documented numbers rather than unverified claims: verified multi-table SQL depth with two real data-quality bugs found and fixed, a validated 84.63% sentiment accuracy, confirmed-relevant multilingual semantic search, a 15/15 text-to-SQL test result with a transparent iteration log, and a single deployed, publicly accessible Streamlit application tying all three layers together, built and shipped despite genuine engineering obstacles (database size limits, hosting-platform changes, vector-store path issues) that were diagnosed and resolved rather than avoided.